SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
Tiruchirappalli Campus — Department of Computer Science & Engineering

ACADEMIC YEAR 2026–2027
2nd Year B.Tech CSE (Core)

EXAMOPT — ADVANCED PROGRAMMING PRACTICE (APP) VIVA QUESTION BANK

50% Academic Review Checkpoint • Theoretical Proofs, Algorithmic Analysis & Code Defense

Team Members:
Suhas V A
Subaash G
Roshan Albert Shaiju

Purpose & Guide: This document compiles the comprehensive question bank and oral defense guide for the 50% Academic Milestone Checkpoint of **ExamOpt** in the *Advanced Programming Practice (APP)* course. Questions are organized into technical pillars reflecting SRM IST evaluation rubrics: Object-Oriented Architecture, Combinatorial Algorithms, Relational Database Engineering, Academic Constraint Auditing, and Python Systems Internals.

PILLAR 1: SOFTWARE ARCHITECTURE & DESIGN PATTERNS

Q1: What software design patterns are implemented in ExamOpt? Where are they in the codebase?

Faculty Evaluation Intent: *Tests if students understand architectural separation of concerns or merely wrote an ad-hoc monolithic script.*

ExamOpt incorporates four core Gang of Four (GoF) / Enterprise design patterns:

- **Facade Pattern (services/exam_service.py):** ExamService acts as a high-level API orchestrator, decoupling UI/CLI clients from database queries, scheduling logic, and constraint validation.
- **Data Access Object (DAO) / Repository Pattern (database/database.py):** All SQLite operations and raw SQL queries are strictly contained in Database. Migrating to PostgreSQL or MySQL requires changes only to this single class.
- **Strategy Pattern (scheduling/room_allocator.py):** RoomAllocator encapsulates the Best-Fit capacity matching strategy independently from Scheduler, allowing dynamic swapping of allocation policies (e.g., First-Fit, Largest-Fit).
- **Specification / Validator Pattern (validation/validator.py):** Academic regulations are isolated into static, pure validation functions returning standardized ValidationResult DTOs.
- **Data Transfer Objects (DTO) (models/*.py):** Pure Python @dataclass structures decouple internal records from persistence frameworks.

Q2: Why did you use Python @dataclass for your models instead of standard classes or dictionaries?

Faculty Evaluation Intent: *Evaluates mastery of modern Python OOP features (PEP 557) and defensive programming.*

Standard dictionaries lack schema enforcement and type checking, leading to runtime KeyError bugs. Conversely, traditional Python classes require verbose boilerplate code for __init__, __repr__, and equality checking.

We utilized @dataclass across all models (Student, Subject, Room, Faculty, TimeSlot, Exam) because:

- **Boilerplate Elimination:** Automatically synthesizes constructor, string representation, and comparison operators.
- **Static Typing Support:** Enforces type hints (e.g., student_count: int = 0, is_available: bool = True).
- **Explicit Serialization:** Every model implements standardized to_dict() and from_dict() methods, enabling two-way translation between SQLite records, GUI widgets, and upcoming TCP socket JSON messages.

Q3: Explain the flow of data when a user clicks 'Generate Timetable' in the GUI.

Faculty Evaluation Intent: *Checks end-to-end trace comprehension across the tiered architecture.*

The data flow strictly adheres to top-down delegation without circular dependencies:

1. **Presentation Layer (gui/views/timetable_view.py):** User clicks the button; the event handler invokes self.service.generate_timetable().

2. **Service Layer (services/exam_service.py):** Extracts active subjects, faculty, rooms, timeslots, and enrollment mappings from Database.

3. **Scheduling Engine (scheduling/scheduler.py):** Sorts subjects by enrollment descending (MRV heuristic) and checks candidate timeslots.

4. **Allocation Strategy (scheduling/room_allocator.py):** Assigns the smallest suitable unbooked hall via Best-Fit.

5. **Constraint Auditing (validation/validator.py):** Performs defensive 6-point validation on candidate assignments.

6. **Persistence & View Update:** The service commits the generated exams to SQLite and updates the GUI Treeview.

PILLAR 2: ALGORITHMS, HEURISTICS & COMPUTATIONAL COMPLEXITY

Q4: What is the computational complexity of the Examination Timetabling Problem (ETP)? Why not use Brute Force?

Faculty Evaluation Intent: *Probes foundational Discrete Mathematics, Graph Theory, and Analysis of Algorithms knowledge.*

The Examination Timetabling Problem is a proven **NP-hard combinatorial optimization problem** (reducible to Graph Vertex Coloring and Bin Packing).

Given S subjects, T timeslots, R rooms, and F faculty members, the brute-force search space is $O((T \times R \times F)^S)$.

For our realistic SRM dataset (6 subjects, 6 slots, 7 rooms, 6 faculty): $(6 \times 7 \times 6)^6 = 252^6 \approx 2.56 \times 10^{14}$ combinations. For an entire university semester (50 subjects), the search space exceeds 10^{100} states, which is mathematically impossible to evaluate before the heat death of the universe.

Hence, ExamOpt employs a **greedy heuristic constraint satisfaction algorithm** that solves the timetable in $O(S \cdot T \cdot R)$ time, terminating in under 10 milliseconds.

Q5: Explain the Most Constrained Variable (MCV / MRV) Heuristic in scheduler.py.

Faculty Evaluation Intent: *Verifies that students understand why their heuristic works rather than just copying search code.*

In Constraint Satisfaction Problems (CSP), the **Most Constrained Variable (MCV)** (also known as the Minimum Remaining Values or *Fail-First* principle) dictates that the variable with the fewest valid options should be assigned first.

In scheduling/scheduler.py, we apply this by sorting subjects descending by enrolled student count:

sorted_subjects = sorted(subjects, key=lambda s: len(students_by_subject[s.id]), reverse=True)

Why this is optimal: High-enrollment courses (e.g., 80 students) require large auditoriums (which are scarce) and create student conflicts across multiple departments. If small classes are scheduled first, they consume large halls or block optimal slots, forcing massive backtracking later. Placing large courses first guarantees early feasibility.

Q6: Explain the Best-Fit Room Allocation algorithm. What is its advantage over First-Fit?

Faculty Evaluation Intent: *Assesses understanding of resource fragmentation and classic bin-packing heuristics.*

Assigning classrooms to exams is an instance of the **Bin Packing Problem**.

- **First-Fit Flaw:** Simply assigns the first available room where $Capacity \ge Students$. If a 15-student elective runs first, First-Fit might allocate the 120-seat Auditorium, stranding a 70-student engineering course later with no hall large enough.
- **Best-Fit Strategy:** Filters unbooked, active rooms where $Capacity \ge Students$, then sorts them ascending by capacity: suitable_rooms.sort(key=lambda r: (r.capacity, r.name)).
- **Mathematical Objective:** Minimizes wasted seat capacity $(Capacity(r) - Students)$ per slot, preserving high-capacity venues for high-capacity cohorts.

```
def allocate_best_fit_room(student_count, slot_id, rooms, schedule):
    available = get_available_rooms_for_slot(slot_id, rooms, schedule)
    suitable = [r for r in available if r.capacity >= student_count]
    if not suitable:
        return None, 'CAPACITY_EXCEEDED'
    suitable.sort(key=lambda r: (r.capacity, r.name)) # Smallest adequate room
    return suitable[0], 'OK'
```

Q7: How is student collision detected? What data structures make this operation fast?

Faculty Evaluation Intent: *Checks if students understand hash-based data structures versus naive quadratic list searches.*

Two exams E_1 and E_2 scheduled in the same timeslot clash if any student is enrolled in both subjects: $\text{Cohort}(E_1) \cap \text{Cohort}(E_2) \ne \emptyset$.

In validation/validator.py, we implement this using **Python Sets (hash tables)**:

current_students_set = set(enrolled_students)
overlap = current_students_set.intersection(set(other_students))

Complexity Comparison:

- Naive nested list iteration: $O(N_1 \times N_2)$ quadratic comparisons.
- Hash Set Intersection: $O(\min(N_1, N_2))$ average time. In a university with thousands of enrollments, set intersection executes in microseconds.

PILLAR 3: DATABASE ENGINEERING & DATA INTEGRITY (SQLITE)

Q8: What Normal Form is your database schema in? Explain your table relationships.

Faculty Evaluation Intent: *Tests core DBMS theory: functional dependencies, normalization, and relational modeling.*

The database schema in database/schema.py is normalized to **3rd Normal Form (3NF)**:

- **1NF:** All attributes are atomic (no repeating groups or multi-valued columns).
- **2NF:** All non-key attributes are fully functionally dependent on the entire primary key.
- **3NF:** No transitive functional dependencies exist ($X \to Y \to Z$).

Relational Decomposition: The many-to-many relationship between students and subjects is resolved via the associative junction table enrollments with a composite primary key PRIMARY KEY (student_id, subject_id).

Q9: Why did you explicitly execute 'PRAGMA foreign_keys = ON;'? Explain your cascade rules.

Faculty Evaluation Intent: *Evaluates practical knowledge of SQLite quirks and referential integrity maintenance.*

By default in SQLite 3, foreign key constraints are disabled for backwards compatibility with legacy applications. To enforce relational integrity, we explicitly execute PRAGMA foreign_keys = ON; upon opening every connection.

Deliberate Cascading Rules:

- enrollments table: FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE CASCADE. If a student withdraws or is deleted, all their subject registrations vanish automatically, preventing orphaned 'ghost' enrollments.
- exams table: FOREIGN KEY (room_id) REFERENCES rooms(room_id) ON DELETE SET NULL. If a physical room is decommissioned, the scheduled exam remains intact but its room_id reverts to NULL, signaling administrators to reallocate a venue.

Q10: Why did you create indices on foreign keys? What is the performance impact?

Faculty Evaluation Intent: *Checks indexing mechanics, B-trees, and query optimization.*

In relational databases, primary keys are indexed automatically using B-Trees, but foreign keys are not.

During timetable validation, the engine repeatedly executes queries such as:
SELECT student_id FROM enrollments WHERE subject_id = ?; and SELECT * FROM exams WHERE slot_id = ?;

Without indices, SQLite must perform a full-table sequential scan ($O(N)$).

We created explicit secondary B-Tree indices in database/schema.py:
• CREATE INDEX idx_enrollments_subject ON enrollments(subject_id);
• CREATE INDEX idx_exams_slot ON exams(slot_id);
This drops query lookups from $O(N)$ full table scans to $O(\log N)$ index searches.

Q11: How do you ensure ACID transactions and protect against SQL Injection?

Faculty Evaluation Intent: *Assesses transaction management, data safety, and secure programming practices.*

- **Atomicity & Consistency (ACID):** In database/database.py, all modifications are wrapped in Python's context manager: with self.get_connection() as conn:. If an exception occurs, the transaction automatically rolls back. If all statements execute successfully, conn.commit() commits all changes atomically.
- **SQL Injection Prevention:** Every SQL command strictly utilizes parameterized queries with ? placeholders (e.g., cursor.execute('SELECT * FROM students WHERE student_id = ?', (student_id,))). String concatenation (f'SELECT ... {id}') is strictly prohibited across the codebase, neutralizing SQL injection vectors.

PILLAR 4: CONSTRAINT VALIDATION & THE 5 MILESTONE CHECKPOINTS

Q12: What are the 5 Milestone 6 Review Scenarios and what does each test verify?

Faculty Evaluation Intent: *Directly tests compliance with the Milestone 6 academic review requirements.*

The 5 mandatory review scenarios in tests/test_scenarios.py verify all hard constraints under isolated harnesses:

- **Test 1 (Valid Data Schedule Generation):** Populates the realistic SRM sample dataset across CSE, IT, and ECE. Asserts that all 6 exams are scheduled and that Validator.validate_full_timetable() returns is_valid == True.
- **Test 2 (Student Conflict Detection):** Assigns two subjects sharing enrolled students (RA03, RA04) to the same slot. Asserts rejection with status ValidationCode.CONFLICT.
- **Test 3 (Capacity Exceeded Detection):** Attempts to schedule 54 students into a 30-capacity lab. Asserts rejection with status ValidationCode.CAPACITY_EXCEEDED and verifies the 24-seat deficit calculation.
- **Test 4 (Duplicate Room Booking):** Books two distinct exams in the same room and same timeslot. Asserts rejection with status ValidationCode.DUPLICATE_BOOKING.
- **Test 5 (Resource Availability Rejection):** Schedules exams with a room under renovation or a professor on medical leave. Asserts rejections with ROOM_UNAVAILABLE and FACULTY_UNAVAILABLE.

Q13: Why do unit tests use an in-memory database (:memory:) instead of the disk database?

Faculty Evaluation Intent: *Evaluates professional test hygiene and isolation.*

In tests/test_crud.py and tests/test_scenarios.py, we instantiate the database with Database(':memory:').

Engineering Advantages:

- **Test Isolation & Idempotence:** Each test runs on a pristine RAM database. Tests cannot contaminate each other or corrupt production examopt.db data.
- **Zero Disk I/O Latency:** Memory reads/writes execute in nanoseconds, allowing our entire 24-test suite to complete in **0.010 seconds**.
- **Automatic Teardown:** Once test execution finishes, RAM is released instantly with no residual scratch files.

PILLAR 5: PYTHON SYSTEMS INTERNALS & THE POST-50% ROADMAP

Q14: Why does your roadmap mandate Multiprocessing instead of Threading for schedule search?

Faculty Evaluation Intent: *The quintessential Python systems interview question: The Global Interpreter Lock (GIL).*

In CPython, the **Global Interpreter Lock (GIL)** is a mutex that prevents multiple native OS threads from executing Python bytecodes simultaneously.

- **I/O-Bound vs. CPU-Bound Tasks:** Threading is effective for network I/O or GUI responsiveness (waiting for user clicks). However, timetable optimization (evaluating thousands of candidate permutations) is purely **CPU-bound mathematical computation**.
- **Multiprocessing Solution:** By deploying Python's multiprocessing module (specifically ProcessPoolExecutor), we spawn separate operating system processes. Each process hosts its own independent Python interpreter and memory space, bypassing the GIL and unlocking 100% parallel computation across all CPU cores.

Q15: How will the TCP Socket Client-Server architecture work in Milestone 7?

Faculty Evaluation Intent: *Checks technical clarity on the upcoming networking milestone.*

Currently, ExamOpt runs as a desktop application. In Milestone 7, we will decouple it into a distributed network architecture:

- **Socket Server (exam_server.py):** A standalone daemon listening on a dedicated TCP port (e.g., 8080) using Python's low-level socket library.
- **Message Protocol:** Requests and responses will be serialized as UTF-8 encoded JSON payloads (e.g., {'command': 'GENERATE_SCHEDULE', 'payload': ...}).
- **Architectural Advantage:** Because we already encapsulated all business logic in ExamService and implemented to_dict()/from_dict() on domain models, the socket server simply unmarshals JSON, delegates to ExamService, and marshals the resulting ScheduleReport back over the network socket.

Q16: How will Functional Programming (map, filter, reduce) be used in Milestone 8?

Faculty Evaluation Intent: *Evaluates functional paradigm application within the course curriculum.*

In Milestone 8, schedule evaluation will be rewritten from imperative loops to pure functional pipelines:

- filter(): Isolate exams violating soft preferences (e.g., extracting students scheduled for consecutive exam slots).
- map(): Calculate mathematical penalty scores or venue occupancy ratios for each scheduled exam.
- functools.reduce(): Aggregate individual penalty scores into a single scalar fitness value for the entire timetable without mutating global state.

Q17: What is the difference between Hard Constraints and Soft Constraints in ExamOpt?

Faculty Evaluation Intent: *Distinguishes binary constraint satisfaction from mathematical objective optimization.*

- **Hard Constraints (Milestones 1–6):** Mandatory physical and institutional laws (binary satisfaction: must have 0 violations). Examples: no student exam clash, no room double-booking, no seating capacity overflow, and no unavailable staff. If any hard constraint fails, the timetable is completely invalid.
- **Soft Constraints (Milestones 10–11):** Desirable academic preferences evaluated via a cost/fitness function. Examples: maximizing rest days between exams for students, balancing invigilation duty hours evenly among professors, and avoiding exams on Saturday afternoons. Timetables are scored and optimized using Genetic Algorithms to minimize total penalty cost.

PILLAR 6: VIVA DEFENSE & RAPID-FIRE TRICK QUESTIONS

Examiner Challenge / Trick Question	Exact Model Response & Code Pointer
'What happens if 500 students enroll in a subject and the largest auditorium seats only 120?'	The RoomAllocator detects that no room satisfies capacity >= 500. It returns (None, 'CAPACITY_EXCEEDED'). The Scheduler marks the exam as Unscheduled with an explicit capacity error rather than crashing or overflowing a room.
'Can two exams happen in the same room at the same time if they are different subjects?'	**Never.** Validator.validate_assignment() explicitly checks other.slot_id == exam.slot_id and other.room_id == exam.room_id and rejects the assignment with DUPLICATE_BOOKING.

'Show me your code handles faculty on leave.'	In scheduling/scheduler.py, we filter available_faculty = [f for f in faculty_list if f.is_available]. Furthermore, Test 5 in tests/test_scenarios.py specifically verifies that assigning an unavailable faculty triggers FACULTY_UNAVAILABLE.
'What was your individual contribution to the project?'	Clearly state your designated module: e.g., *'I engineered the 3NF SQLite schema, DAO layer, and transactional CRUD operations in database/'*, or *'I designed the 6-point constraint validation engine and automated test suites in validation/ and tests/'*, or *'I implemented the MRV heuristic scheduler and Best-Fit room allocation in scheduling/'*.

APPENDIX: TERMINAL COMMANDS CHEATSHEET

Command Line Instruction	Description / Viva Demonstration Purpose
`python main.py`	Launches the Modern Obsidian Studio desktop interface (Default).
`python main.py --theme classic`	Launches the Classic University Tabbed Notebook desktop interface.
`python main.py --cli`	Executes full end-to-end demonstration in CLI mode (ideal for headless lab PCs).
`python main.py --scenarios`	Directly runs and evaluates the 5 mandatory Milestone 6 academic scenarios.
`python -m unittest discover tests -v`	Runs the complete 24-test automated verification suite (~0.010s execution).